

RAG-based Intelligent Rental Consultation System

Project Information

Team Information

- **Group Number:** Group 10
- **Team Members:** Meng Zihao, Xue Wentao, Liu Junyou, Dong Liang

Project Links

- **GitHub Repository**
 - Backend: <https://github.com/M-Downey/5105-Capstone-backend>
 - Frontend: <https://github.com/M-Downey/5105-Capstone-frontend>
 - **Demo**
 - <http://downey.top/login>
-

1. Problem Statement

In the era of information explosion, enterprises and individuals face challenges in managing massive documents and knowledge retrieval. Traditional document management systems lack intelligent Q&A capabilities, while general-purpose large language models are prone to "hallucination" issues.

This project builds an intelligent chat system based on RAG (Retrieval-Augmented Generation) technology, targeting the Singapore rental consultation domain. Key features:

1. **Knowledge Base Management:** Upload and manage documents (PDF, Word, text, etc.) with automatic parsing and vector indexing
 2. **Intelligent Retrieval:** Semantic similarity-based vector retrieval to locate relevant document segments
 3. **Context-Enhanced Generation:** Generate accurate answers using retrieved context and historical conversations
 4. **User-Friendly Interface:** Modern web interface with real-time streaming responses
-

2. Architecture

2.1 Frontend Architecture

The frontend uses React technology stack to build a responsive single-page application (SPA).

Technology Stack

- **React 19:** UI framework with Hooks for state management
- **TypeScript:** Type safety
- **Vite:** Fast build tool
- **Tailwind CSS:** Utility-first CSS framework
- **React Router:** Client-side routing

- **Axios:** HTTP client
- **React Markdown:** Markdown rendering
- **i18next:** Internationalization (Chinese/English)

Core Modules

1. Routing and Navigation (App.tsx)

- Route guards for authentication
- Unified Header with navigation and language switching
- Responsive layout

2. Authentication Module (LoginPage.tsx, RegisterPage.tsx)

- User registration (regular users and administrators)
- JWT Token authentication
- Automatic token refresh and error handling

3. Chat Module (ChatPage.tsx)

- Conversation management: Create new conversations, list all conversations, switch between conversations
- Message history: Load and display historical conversation records chronologically
- Real-time interaction:
 - Supports both streaming response (SSE) and regular response modes
 - Typewriter effect: Character-by-character display of AI responses
 - Auto-scroll to bottom for latest messages
- Markdown rendering: AI responses support Markdown format (code blocks, lists, quotes)
- Reference document display: Shows referenced document list at the end of AI responses

4. Document Management Module (DocsPage.tsx)

- Document upload: Supports drag-and-drop or click upload, multiple formats (PDF, TXT, MD, DOC, DOCX, HTML)
- Document list: Displays all uploaded documents with title, filename, size information
- Document preview: Modal preview of document content (supports PDF, Word, text formats)
- Document deletion: Administrators can delete documents (removes from both file system and vector index)

Frontend Architecture Characteristics

- Component-based design: Highly modular, easy to maintain and extend
- State management: React Hooks for local state management
- Error handling: Unified error notification mechanism (Sonner Toast)
- Performance optimization: Lazy loading, code splitting, caching strategies

2.2 Backend Engineering Architecture

The backend adopts Spring Boot + SSM (Spring + SpringMVC + MyBatis) architecture.

2.2.1 User Module

Core Components

1. UserService (User Service Layer)

- `register(String username, String password, boolean isAdmin) : User` registration
 - Username uniqueness validation
 - Password encrypted using BCrypt
 - Supports both administrator and regular user roles
- `login(String username, String password) : User` login
 - Validates username and password
 - Generates and returns JWT Token to client

2. AuthController (Authentication Controller)

- `POST /api/auth/register` : User registration endpoint
- `POST /api/auth/login` : User login endpoint, returns JWT Token

3. JWT Authentication Mechanism

- **JwtService**: Token generation and parsing
 - Uses HS256 algorithm for signing
 - Token contains username and role information (isAdmin)
 - Configurable expiration time (default 120 minutes)
- **JwtAuthenticationFilter**: Request interceptor
 - Extracts Bearer Token from request header
 - Validates token validity
 - Injects user information into Spring Security context

- 4. **SecurityConfig**: Configures Spring Security filter chain, JWT authentication with stateless session management

Data Model

- **User Table**: id (primary key), username (unique), password_hash, is_admin, created_at

Implementation Principle

1. During user registration, passwords are encrypted using BCryptPasswordEncoder before storage
2. During user login, the system validates the password and generates a JWT Token
3. Subsequent requests carry the Token, JwtAuthenticationFilter validates the Token and sets authentication information
4. Uses `@PreAuthorize("hasRole('ADMIN')")` annotation to implement role-based access control

2.2.2 Knowledge Base Module

Core Components

1. DocumentService (Document Service Layer)

- `upload(Long userId, String title, MultipartFile file)` : Document upload
 - File storage: Uses UUID to generate unique filenames, avoiding conflicts
 - Database records: Saves document metadata (title, filename, type, size, storage path)
 - Automatic indexing: Calls corresponding parsing and indexing methods based on file type
- `delete(Long id)` : Document deletion (removes from file system and database)
- `list()` : Retrieves all document list
- `getDocumentContent(Long id)` : Retrieves original document content for preview

2. DocumentController (Document Controller)

- `POST /api/docs/upload` : Upload document (admin only)
- `DELETE /api/docs/{id}` : Delete document (admin only)
- `GET /api/docs` : Get document list (admin only)
- `GET /api/docs/{id}` : Get document details (admin only)
- `GET /api/docs/{id}/content` : Get document content (admin only)

3. Multi-Format Document Parsing

- **PDF Parsing**: Uses Apache PDFBox library to extract text content
- **Word Parsing**: Uses Apache POI library, supports .doc (HWPF) and .docx (XWPF) formats
- **Text Files**: Direct reading (TXT, MD, HTML)
- **HTML Parsing**: Simple tag removal, extracts text content

Document Indexing Process

Document Upload → File Storage → Document Parsing → Text Extraction → Document Splitting → Vectorization → Store to Vector Database

Data Model

- **Document Table**: id, title, filename, content_type, size_bytes, storage_path, created_by, created_at

Implementation Principle

1. After document upload, system determines file type based on file extension and Content-Type
2. Calls corresponding parser to extract text content
3. Uses LangChain4j's DocumentSplitter to split documents into smaller text segments (chunks)
 - Splitting strategy: Recursive splitting, chunk size 1000 characters, overlap 100 characters
 - Preserves document metadata (filename, file type) in each chunk

4. Uses OpenAI's embedding model (text-embedding-3-small) to convert text segments into vectors
5. Stores vectors in InMemoryEmbeddingStore (in-memory vector database)

Index Rebuilding on Startup

- RagBootstrap component automatically scans upload directory when application starts
 - Re-indexes all existing documents to ensure vector index synchronization with file system
-

2.3 Backend Algorithm Architecture

2.3.1 Chat Module

Function Overview The chat module is responsible for conversation management, message storage, and historical record maintenance, providing conversation context for the RAG system.

Core Components

1. ChatService (Chat Service Layer)

- `createChat(Long userId, String title)` : Create new conversation
- `listChats(Long userId)` : Get all conversation list for user
- `history(Long chatId)` : Get historical messages for conversation
- `userSend(Long chatId, String content)` : Save user message
- `aiReply(Long chatId, String userContent)` : Generate and save AI reply (non-streaming)
- `aiReplySave(Long chatId, String answer)` : Save AI reply (streaming scenario)

2. ChatController (Chat Controller)

- `POST /api/chat/create` : Create conversation
- `GET /api/chat/list` : Get conversation list
- `GET /api/chat/{chatId}/history` : Get historical messages
- `POST /api/chat/{chatId}/send` : Send message (non-streaming)
- `POST /api/chat/{chatId}/stream` : Send message (streaming, SSE)

Data Model

- **Chat Table:** id (primary key), user_id (foreign key), title, created_at
- **Message Table:** id (primary key), chat_id (foreign key, indexed), role ("user" or "assistant"), content, created_at

Implementation Principle

1. Each user can have multiple conversations, each conversation contains multiple messages
2. Messages are stored in chronological order, supporting fast queries by chat_id
3. Streaming response uses Server-Sent Events (SSE) technology:
 - Frontend sends POST request to `/api/chat/{chatId}/stream`
 - Backend creates SseEmitter with 5-minute timeout
 - Asynchronously calls RAG service to generate streaming reply

- Each token is sent to frontend in real-time via SSE events
- After generation completes, sends "done" event and closes connection

2.3.2 RAG Logic

Function Overview RAG (Retrieval-Augmented Generation) is the core algorithm module, responsible for retrieving relevant information from the knowledge base and combining it with large language models to generate accurate answers.

Core Components

1. RagService (RAG Service Layer)

- **Document Indexing Methods:** `indexPdf()` , `indexText()` , `indexWord()` , `indexHtml()`
- **Core Methods:**
 - `chatWithRag(Long chatId, String userMessage)` : Non-streaming RAG chat
 - `chatWithRagStreaming(Long chatId, String userMessage, StreamingResponseHandler handler)` : Streaming RAG chat
 - `getRagReferences(String userMessage)` : Get RAG referenced document list
 - `buildRagContextWithReferences(String userMessage)` : Build RAG context and collect references

2. RagConfig (RAG Configuration)

- Configures OpenAI Chat model (gpt-4o-mini)
- Configures OpenAI Streaming Chat model
- Configures OpenAI Embedding model (text-embedding-3-small)
- Configures vector storage (InMemoryEmbeddingStore)

RAG Workflow

```
User Question
↓
Question Vectorization (Embedding)
↓
Vector Similarity Retrieval (Top-K, K=4)
↓
Retrieve Relevant Document Segments
↓
Build System Prompt (including retrieved context)
↓
Load Historical Conversation (last 10 messages)
↓
Build Complete Message List (System + History + User)
↓
```

Call LLM to Generate Answer

↓

Return Answer (streaming or non-streaming)

Detailed Implementation

1. **Question Vectorization:** Uses OpenAI's text-embedding-3-small model to convert user question to 1536-dimensional vector
2. **Vector Retrieval:** Finds 4 most similar document segments using cosine similarity calculation
3. **Context Construction:** Concatenates retrieved document segments into context, collects referenced document names
4. **Historical Context Management:** Loads last 10 historical messages from database, converts to ChatMessage format
5. **Message List Construction:** Builds complete message list (SystemMessage + History + UserMessage)
6. **LLM Generation:** Calls OpenAI model (streaming or non-streaming) to generate answer

Key Technical Points

1. **Document Splitting Strategy:** Recursive splitter, chunk size 1000 characters, overlap 100 characters, preserves document metadata
2. **Context Window Management:** Limits historical message count (MAX_HISTORY_MESSAGES = 10) to avoid token overflow
3. **Reference Document Tracking:** Adds "source" metadata during splitting, extracts document name during retrieval, displays at end of answer
4. **Error Handling:** Document parsing failures don't affect file upload, indexing failures log warnings, don't prevent application startup

2.4 System Deployment Architecture

Container Architecture

- **MySQL 8.0:** Relational database (UTF8MB4, persisted to `/data/mysql`)
- **Backend:** Spring Boot (port 8080, multi-stage Docker build)
- **Nginx:** Reverse proxy and static file server (port 80)

Deployment Process

1. Environment Preparation

- Create `.env` file, configure necessary environment variables (OPENAI_API_KEY, database credentials)
- Build frontend: `cd frontend && npm ci && npm run build`
- Copy frontend build artifacts to `frontend-dist/` directory

2. One-Click Startup

```
docker compose up -d --build
```

- Automatically pulls MySQL image
- Builds backend image (multi-stage Docker build)
- Starts all services (MySQL, Backend, Nginx)

3. Data Persistence

- MySQL data: `./data/mysql` (persisted to host)
- Uploaded files: `./data/uploads` (document storage)
- Log files: `./data/logs/app.log` (daily rotation)

Network Architecture



Configuration Management

- **Environment Variables:** Managed through `.env` file for sensitive configuration
- **Application Configuration:** `application.yml` (can be overridden by environment variables)
- **Log Configuration:** `logback-spring.xml` , supports daily rotation
- **Database Migration:** Flyway automatically executes SQL migration scripts

3. Tech Stack

3.1 Frontend

React, TypeScript, Vite, Tailwind CSS, React Router, Axios, React Markdown, i18next

3.2 Backend

Java 17, Spring Boot, Spring Security, MyBatis, MySQL, Flyway, JWT, LangChain4j, Apache PDFBox, Apache POI

3.3 AI/ML

OpenAI API, GPT-4o, text-embedding-3-small, LangChain4j, InMemoryEmbeddingStore

3.4 Deployment

Docker, Docker Compose, Nginx, Maven

4. Features

4.1 User Authentication and Authorization

-  User registration and login (JWT Token, BCrypt encryption)
-  Role-based access control (RBAC)
-  Automatic token refresh and expiration handling

4.2 Document Management

-  Multi-format upload (PDF, TXT, MD, HTML, DOC, DOCX)
-  Automatic parsing and vector indexing
-  Document preview and deletion
-  Metadata management

4.3 Intelligent Q&A

-  RAG-based Q&A with semantic similarity retrieval
-  Context-aware conversation (historical message memory)
-  Streaming response (SSE) with reference document tracking
-  Markdown format support and multi-conversation management

4.4 User Experience

-  Modern UI (Tailwind CSS), responsive layout
-  Real-time typewriter effect, auto-scroll
-  Internationalization (Chinese/English)
-  Error notifications and loading states

4.5 System Features

-  Containerized deployment (Docker Compose)
 -  Data persistence and logging
 -  Automatic index rebuilding on startup
-

5. Insights

5.1 Technical Insights

1. RAG Architecture Advantages

- Solves LLM hallucination by retrieving real documents
- Supports dynamic knowledge base updates without retraining
- Strong explainability with document source references

2. Vector Retrieval Precision

- Semantic similarity over keyword matching
- Top-K strategy (K=4) balances accuracy and context length
- Document splitting (1000 chars, 100 overlap) ensures context completeness

3. Context Management

- Limits to 10 historical messages to avoid token overflow
- System prompt includes RAG context for knowledge-based answers

4. Streaming Response

- SSE enables true streaming transmission

- Asynchronous processing improves concurrency

5.2 Architecture Design Insights

1. **Layered Architecture:** Clear separation (Controller, Service, Mapper, Domain)
2. **Security Design:** JWT stateless authentication, password encryption, RBAC
3. **Scalability:** Pluggable parsers, abstracted vector storage, externalized configuration

5.3 Performance Optimization

1. **Index Rebuilding:** Automatic on startup with exception handling
 2. **Database Indexing:** chat_id index for message queries
 3. **Frontend Optimization:** Code splitting, lazy loading, optimistic updates
-

6. Challenges

6.1 Technical Challenges

1. Vector Storage Limitations

- **Problem:** InMemoryEmbeddingStore loses index on restart
- **Solution:** Automatic index rebuilding; future migration to persistent vector DB

2. Document Parsing Complexity

- **Problem:** Different formats require different parsers
- **Solution:** Multiple parsers implemented; graceful error handling

3. Context Window Management

- **Problem:** Balance between context length and token limits
- **Solution:** Limit to 10 messages; dynamic index calculation

4. Streaming Response Stability

- **Problem:** SSE connections may be interrupted
- **Solution:** Timeout mechanism (5 min), error handling, retry logic

6.2 Business Challenges

1. **Document Quality Impact:** Document preview for quality checking
2. **Concurrent Access:** Thread pool and connection pool management
3. **Scale Expansion:** Future migration to professional vector database

6.3 Deployment Challenges

1. **Environment Configuration:** .env file for unified management
 2. **Data Migration:** Flyway for version management, persistent storage
-

7. Conclusion

This project successfully builds an intelligent Q&A system based on RAG technology, implementing a complete workflow from document management to intelligent Q&A.

7.1 Project Achievements

1. **Functional Completeness:** User authentication, document management, intelligent Q&A
2. **Technical Advancement:** RAG architecture with vector retrieval and LLM integration
3. **User Experience:** Modern web interface with streaming response and internationalization
4. **Deployability:** Containerized deployment, one-click startup

7.2 Technical Highlights

1. **RAG Architecture:** Solves hallucination problem, provides explainable answers
2. **Context Awareness:** Combines historical conversations and knowledge base context
3. **Multi-Format Support:** Automatic parsing and indexing of multiple document formats
4. **Streaming Response:** Real-time streaming generation

7.3 Future Improvements

1. **Vector Storage:** Migrate to persistent vector database (Pinecone, Weaviate)
2. **Document Parsing:** Introduce more powerful libraries (Apache Tika)
3. **Performance:** Caching mechanisms, asynchronous task queues
4. **Features:** Document versioning, multi-knowledge base isolation, full-text search
5. **Monitoring:** Health checks, performance monitoring, log analysis

7.4 Summary

This project demonstrates the practical application of RAG technology in knowledge Q&A. Through reasonable architecture design and technology selection, it builds a functional, performant, and easily deployable intelligent Q&A system that solves traditional document management pain points and provides new tools for enterprise knowledge management.